

서비스 구성 예시



서비스 이해를 위한 예시 웹툰



'당신의 목소리로 다음 글을 읽어주세요(당목담글)' 프로젝트

멀티 에이전트 시스템(Multi-Agent System) 설계

효율성과 전문성을 극대화하기 위해 각 에이전트별로 성격이 다른 최신 LLM(대규모 언어 모델)을 배치
단, 각 단계에서 비용 부담 없이 강력한 성능을 낼 수 있는 '무료/오픈소스 기반 멀티에이전트 시스템'
을 설계

 무료 모델 기반 멀티에이전트 아키텍처

에이전트 구분	추천 모델 (무료/오픈소스)	역할 및 특징
1. RAG 수집	Grok (xAI)	실시간 웹 검색 능력이 탁월하여 최신 수필 데이터나 상황 정보(뉴스, 트렌드)를 가장 빠르게 수집.
2. 벡터 DB 저장	Llama 3.1 8B (via Groq)	ChromaDB와 연동하여 텍스트를 정밀하게 분할하고 인덱싱합니다. Groq 엔진을 사용하면 속도가 매우 빠르다.
3. 대본 생성	DeepSeek-V3	문학적 표현과 감성적 코딩 능력이 우수하여, 상황에 맞는 따뜻한 이어읽기 대본과 '마중물' 문구를 작성.
4. 이미지 생성	Flux.1 [schnell]	현재 무료로 사용 가능한 이미지 모델 중 퀄리티가 가장 높습니다. 시의 분위기에 맞는 고해상도 배경을 생성.
5. 요약 생성	Gemini 1.5 Flash	컨텍스트 윈도우가 매우 넓어, 방대한 양의 가족 대화나 수필 전체를 한눈에 파악하고 요약하는 데 최적.
6. PPT 생성	Qwen 2.5 (72B)	코딩 및 논리적 구조 설계 능력이 뛰어납니다. python-pptx 라이브러리를 활용한 슬라이드 자동 생성 코드 작성이 가능.
7. 예상질문 생성	Command R (Cohere)	RAG 및 비즈니스 분석에 특화된 모델로, 기술적 허점이나 서비스 개선 방향에 대한 날카로운 질문을 추출.

1. 데이터 파이프라인 (RAG & Vector DB)

- **RAG 수집 (Grok):** "최근 1년 내 발표된 위로와 공감을 주제로 한 짧은 수필 10편 찾아줘"와 같은 실시간 검색 쿼리를 수행.
- **저장 (Chroma + Llama 3.1):** 수집된 데이터를 **Flowise**나 **LangFlow** 같은 시각화 툴을 통해 **ChromaDB**에 임베딩. 이때 Llama 3.1은 데이터의 메타데이터(작가, 감성 태그 등)를 분류하는 역할을 담당.

2. 콘텐츠 생성 레이어

- **대본 & 마중물 (DeepSeek-V3):** "사용자의 숨소리가 섞인 거친 환경에서도 따뜻하게 들릴 수 있는 릴레이 시의 첫 구절을 작성해"라는 페르소나 기반의 지시를 수행.
- **시각화 (Flux.1):** 생성된 대본의 핵심 키워드를 추출하여 초대장이나 앱 배경 화면에 쓰일 이미지를 생성.

3. 출력 및 검증 레이어

- **슬라이드 자동화 (Qwen 2.5):** 서비스 기획서나 특허 설명 자료를 PPT로 즉시 변환할 수 있도록 구조화된 Markdown 혹은 Python 코드를 출력.
- **피드백 루프 (Command R):** "만약 가족 구성원이 이어읽기를 거부한다면?" 혹은 "음성 병합 시 노이즈가 너무 심하다면?"과 같은 실질적인 운영 질문을 생성하여 시스템을 보완.

💡 실행을 위한 팁 (Orchestration)

이 에이전트들을 하나로 묶기 위해 다음과 같은 **무료 오케스트레이션 도구**를 추천.

- **Flowise / LangFlow:** 코딩 없이 드래그 앤 드롭으로 위 모델들을 연결하여 API로 배포 가능.
- **Ollama (Brew 기반 설치):** Llama 3.1이나 Qwen 같은 모델을 본인의 로컬 PC에서 직접 구동하여 API 비용을 '0원'으로 만들 수 있다.
- **Groq Cloud:** 위에서 언급한 대부분의 오픈소스 모델(Llama, Mixtral 등)을 매우 빠른 속도로 무료 티어 내에서 사용할 수 있다.

멀티에이전트 아키텍처 및 작업 순서

이 시스템은 삼성카드의 사회공헌(ESG) 플랫폼 내에서 '목소리 기부' 및 '가족 연결' 서비스로 확장 가능한 구조.

1. **데이터 파이프라인:** Grok으로 최신 수필/시를 수집하고 Llama 3.1로 **ChromaDB**에 저장.
2. **콘텐츠 생성:** DeepSeek-V3가 감성 대본을 쓰고, Flux가 시각 자료를 생성.
3. **출력 및 검증:** Qwen이 제안용 PPT를 만들고 Command R이 협업 미팅을 위한 예상 질문 추출.

LangGraph 기반 실제 실행 코드

1. 환경 설정 및 라이브러리 설치

```
(pylocal) PS C:\Users\user\Downloads\dangmok\pylocal> pip install langgraph langchain langchain-community chromadb groq google-generativeai cohere python-pptx
Installing collected packages: XlsxWriter, types-requests, lxml, fastavro, python-pptx, cohere
Successfully installed XlsxWriter-3.2.9 cohere-5.21.1 fastavro-1.12.2 lxml-6.1.0 python-pptx-1.0.2 types-requests-2.33.0.20260503
(pylocal) PS C:\Users\user\Downloads\dangmok\pylocal>
pip install langgraph langchain langchain-community chromadb groq google-generativeai cohere
python-pptx
```

2. 에이전트 상태 및 노드 정의

이 코드는 각 단계별 에이전트의 역할을 함수로 정의하고 LangGraph의 Workflow로 연결

```
import os
from typing import TypedDict, List
from langgraph.graph import StateGraph, END
```

```
# [State 정의] 에이전트 간 공유될 데이터 구조
class AgentState(TypedDict):
    topic: str          # 사용자가 던지는 주제
    context: str         # RAG 수집 데이터
    script: str          # 생성된 대본
    image_url: str       # 생성된 이미지 경로/설명
    summary: str         # 요약 내용
    ppt_outline: List[str] # PPT 슬라이드 구성
    questions: List[str] # 예상 질문
```

```
# --- [1. 데이터 파이프라인 에이전트] ---
# RAG 수집
def rag_collection_agent(state: AgentState):
    """Grok (xAI) 모델을 가정한 RAG 수집 (Web Search API 연동)"""
    topic = state['topic']
    # 실제 구현 시 Grok API 호출
    collected_data = f'{topic}과 관련된 최신 수필 및 삼성카드 ESG 캠페인 사례 수집 완료'
    return {"context": collected_data}
```

```
# ChromaDB 저장
def vector_db_agent(state: AgentState):
    """Llama 3.1 (via Groq) + ChromaDB 저장"""
    context = state['context']
    # ChromaDB에 저장하는 로직 (Stitch Engine용 인덱싱)
    print(f"--- 벡터 DB(Chroma)에 데이터 인덱싱 중: {context[:20]}... ---")
    return {"context": context}
```

```
# --- [2. 콘텐츠 생성 레이어 에이전트] ---
# 감성 대본 및 '마중물' 문구 생성
def script_generation_agent(state: AgentState):
    """DeepSeek-V3를 활용한 감성 대본 및 '마중물' 문구 생성"""
    # 상황 인지(소방관 페르소나 등) 로직 포함
    script = "상황: 출동 전 긴박함 속의 따뜻함. 대본: '동료의 숨소리가 들리는 이 길 위에서, 다음 구절을 읽어주세요...'"
    return {"script": script}
```

```
# 시각적 배경 생성
def image_generation_agent(state: AgentState):
    """Flux.1 모델을 활용한 시각적 배경 생성"""
    script = state['script']
    image_prompt = f"Warm and emotional background for:{script}"
    return {"image_url": "generated_image_path.png"}
```

```
# --- [3. 출력 및 검증 레이어 에이전트] ---
# 빠른 요약
def summary_agent(state: AgentState):
    """Gemini 1.5 Flash를 활용한 빠른 요약"""
    script = state['script']
    summary = "가족의 목소리를 잇는 따뜻한 릴레이 서비스 핵심 요약"
    return {"summary": summary}
```

```
# 삼성카드 형업 제안용 PPT 구조 생성
def ppt_generation_agent(state: AgentState):
    """Qwen 2.5를 활용한 삼성카드 형업 제안용 PPT 구조 생성"""
    ppt_outline = ["1. 프로젝트 개요", "2. 삼성카드 결제 데이터 기반 맞춤형 문구", "3. 목소리 기부 프로세스"]
    return {"ppt_outline": ppt_outline}
```

```
# 비즈니스 검증 및 예상 질문
def question_generation_agent(state: AgentState):
    """Command R (Cohere)를 활용한 비즈니스 검증 및 예상 질문"""
    questions = ["Q1. 보안상 음성 데이터의 온디바이스 처리는 어떻게 되나요?", "Q2. 삼성카드 앱과의 API 연동 방식은?"]
    return {"questions": questions}
```

```
# --- [LangGraph 워크플로우 구성] ---

workflow = StateGraph(AgentState)
```



```
# 노드 추가
workflow.add_node("collection", rag_collection_agent)
workflow.add_node("vector_store", vector_db_agent)
workflow.add_node("script_gen", script_generation_agent)
workflow.add_node("image_gen", image_generation_agent)
workflow.add_node("summary", summary_agent)
workflow.add_node("ppt_gen", ppt_generation_agent)
workflow.add_node("qa_gen", question_generation_agent)
```

```
# 에지 연결 (순서대로 실행)
workflow.set_entry_point("collection")
workflow.add_edge("collection", "vector_store")
workflow.add_edge("vector_store", "script_gen")
workflow.add_edge("script_gen", "image_gen")
workflow.add_edge("image_gen", "summary")
workflow.add_edge("summary", "ppt_gen")
workflow.add_edge("ppt_gen", "qa_gen")
workflow.add_edge("qa_gen", END)
```

```
# 컴파일
app = workflow.compile()
```

3. 실제 실행 및 결과 출력

```
# 입력 데이터 설정
inputs = {"topic": "소방관의 일상과 가족에게 전하는 위로의 메시지"}
```

```
# 시스템 실행
for output in app.stream(inputs):
    for key, value in output.items():
        print(f"[{key}] 단계 완료")
```

```
# 최종 결과 확인
final_state = app.invoke(inputs)
print("\n" + "="*50)
print("🚀 삼성카드 협업 제안서 및 에이전트 작업 결과물")
print(f"1. 생성 대본: {final_state['script']}")
print(f"2. PPT 구성: {final_state['ppt_outline']}")
print(f"3. 미팅 대비 질문: {final_state['questions']}")
print("="*50)
```

이 멀티에이전트 시스템은 단순히 텍스트를 만드는 것을 넘어, 협업 시 아래와 같은 **기술적 우위**를 점할 수 있게 설계되었다.

- **개인정보 주권:** Vector DB 에이전트가 온디바이스(Local) ChromaDB를 우선하도록 설정하여, 카드사의 민감한 고객 데이터 외부 유출 우려를 불식시킨다.
- **상황 인지 필터링:** Script 에이전트가 단순 텍스트가 아닌, 사용자의 GPS나 스케줄 정보를 '마중물' 문구에 반영하여 서비스의 독창성(Originality)을 높인다.
- **비용 효율성:** Gemini Flash와 Groq 기반 오픈소스 모델을 섞어 사용하여 대규모 사용자가 유입되어도 API 운영 비용을 획기적으로 낮출 수 있다.

 Flowise 시각화 구성을 위한 3단계 세팅

1. 환경 준비 (빠른 실행)

로컬 환경이나 클라우드에 Flowise를 설치

2. 노드 배치 가이드 (캔버스에 배치할 항목)

Flowise의 각 카테고리에서 아래 노드들을 캔버스에 끌어다 놓으세요.

Flowise 카테고리	노드 이름	실제 역할 (당목담글 프로젝트 매핑)
Multi Agents	Supervisor	전체 흐름을 관장하는 컨트롤러 (삼성카드 ESG 미션 관리)
Multi Agents	Worker (7개)	위에서 설계한 7개의 에이전트 각각 생성
Chat Models	ChatGroq / ChatGoogleGenerativeAI	각 Worker에 연결할 무료 API 모델 (Llama 3, Gemini 등)
Vector Stores	Chroma	스티치 엔진의 지식 창고 (RAG 데이터 저장소)
Tools	Custom Tool (Image, PPT)	이미지 생성 및 PPT 생성을 위한 API 실행 도구

3. 에이전트별 시스템 프롬프트(System Prompt) 설정

각 **Worker** 노드의 System Prompt에 아래 내용을 입력하면 전문가 수준의 답변이 나온다.

- **RAG & Vector Worker:** "너는 수필과 감성 문구를 전문적으로 수집하는 에이전트야. 삼성카드의 따뜻한 브랜드 이미지에 맞는 문장만 선별해."
- **Script Worker:** "사용자의 상황(긴급 직업군 등)을 감지하여 목소리 기부를 위한 '마중물' 대본을 작성해. 따뜻하고 연결되는 느낌이 핵심이야."

- **PPT & Question Worker:** "삼성카드 ESG 부서 미팅용 슬라이드 개요와 예상 질문을 생성해. 보안과 데이터 주권(On-device)을 강조해."

삼성카드 시연용 '와우 포인트(Wow Points)' 연출법

시연 시 단순히 채팅창에 글자를 입력하는 것이 아니라, Flowise의 기능을 활용해 다음을 강조.

① 워크플로우 가시화 (The Flow)

담당자에게 "에이전트들이 서로 어떻게 대화하며 결과물을 완성하는지"를 실시간 그래프 화면으로 보여준다.

"보십시오. 사용자가 주제를 던지면, RAG 에이전트가 데이터를 찾고, 스크립트 에이전트가 감성을 더해 대본을 완성하는 과정이 투명하게 관리됩니다."

② API 연동의 유연성

"현재는 무료 모델(Groq, Gemini)을 쓰고 있지만, 삼성카드의 자체 LLM이나 폐쇄형 GPU 서버로 즉시 교체 가능하다"는 점을 언급. (Flowise는 모델 노드만 갈아 끼우면 됩니다.)

③ 실시간 결과물 확인

Flowise 우측 하단의 '**Chat**' 버튼을 눌러 실제 동작을 시연.

- **입력 예시:** "소방관 아버지가 딸에게 남기는 따뜻한 이어읽기 캠페인을 기획해줘."
- **출력 확인:** 대본 생성 → 이미지 생성 가이드 → PPT 개요가 순차적으로 출력되는 것을 확인시켜 줍니다.

실무 팁: 시연 전 준비물

1. **API Keys:** Groq(무료), Google Gemini(무료), Cohere(무료) 키를 미리 설정 메뉴에 등록.
2. **Custom Tool 코드:** PPT 생성 등이 복잡하다면, 텍스트 형태로 구조화된 결과만 나오게 설정해도 시연용으로는 충분히 강력.
- Cohere : 기업용 생성형 AI와 자연어 처리(NLP)를 위한 플랫폼

Flowise 노드 배치 설계도

 node-v24.15.0-x64.msi : <https://nodejs.org/ko/download>


```
● PS C:\Users\user\Downloads\dangmok\pylocal> node -v
v24.15.0
● PS C:\Users\user\Downloads\dangmok\pylocal> npm -v
11.12.1
● PS C:\Users\user\Downloads\dangmok\pylocal> ./Scripts/activate
● (pylocal) PS C:\Users\user\Downloads\dangmok\pylocal> npm -v
11.12.1
● (pylocal) PS C:\Users\user\Downloads\dangmok\pylocal> node -v
v24.15.0
○ (pylocal) PS C:\Users\user\Downloads\dangmok\pylocal> npx flowise start
Need to install the following packages:
flowise@1.6.0
Ok to proceed? (y) y
```

Flowise의 **Multi Agents** 캔버스에 아래 순서대로 노드를 배치하고 연결.

1. 중앙 통제부 (The Brain)

- **노드:** Supervisor
- **역할:** 사용자의 요청(예: "소방관 테마의 목소리 기부 캠페인 기획")을 받아 각 에이전트에게 업무를 배분합니다.
- **모델:** ChatGroq (Llama 3.1 70B 권장 - 판단력이 뛰어납니다.)

2. 기술 및 콘텐츠 에이전트 (The Workers)

각 에이전트는 Worker 노드로 생성하며, 개별 System Prompt를 설정.

에이전트명	추천 모델 (무료 API)	시스템 프롬프트 핵심 내용 (System Prompt)
RAG & Vector	Grok (xAI)	"삼성카드 ESG 사례와 공감 수필 데이터를 검색하고, 그 핵심을 ChromaDB 지식 창고에 연동해."
감성 대본	DeepSeek-V3	"상황 인지 필터를 적용해. 거친 숨소리와 주변 소음이 보존된 상황에 어울리는 따뜻한 '마중물' 문구를 작성해."
이미지 생성	Flux.1 (Custom Tool)	"대본의 정서(따뜻함, 연결)를 분석하여 시각적 배경을 위한 정교한 프롬프트를 생성해."
요약 및 PPT	Gemini 1.5 Flash	"전체 기획 내용을 요약하고, 삼성카드 제안용 슬라이드 5장 분량의 개요를 python-pptx 구조로 설계해."
비즈니스 검증	Command R	"특히 요건인 '스티치 엔진'의 차별성을 강조하고, 카드사 보안 담당자가 던질 법한 날카로운 질문 3개를 뽑아."

 실제 시연을 위한 에이전트 간 '연결 흐름' (Flow)

1. **Input:** Chat Input 노드에 주제 입력.
2. **Processing:** Supervisor가 RAG 에이전트를 먼저 깨워 데이터를 가져오게 합니다.
3. **Creation:** 가져온 데이터를 바탕으로 대본 → 이미지 에이전트가 결과물을 생성합니다.

4. **Finalize:** PPT와 검증 에이전트가 마무리하여 사용자에게 최종 제안서 형태의 답변을 전달합니다.

삼성카드 협업 및 '당목담글' 프로젝트의 성공적인 시연을 위해, 외부 도구 도움 없이 순수 파이썬 코드 내에서 즉시 호출할 수 있는 에이전트별 상세 시스템 프롬프트(Persona)를 설계

이 프롬프트들은 각 모델의 특성(감성, 논리, 구조화)에 최적화되어 있으며, 삼성카드의 ESG 가치와 프로젝트의 특허 요건을 포함

당목담글 멀티에이전트 시스템 프롬프트 정의 (Python)

```
AGENTS_CONFIG = {  
    # 1. RAG & Vector 에이전트 (The Librarian)  
    "RAG_VECTOR_AGENT": {  
        "model": "Grok-1 / Llama-3.1",  
        "system_prompt": """  
        당신은 '삼성카드 ESG 목소리 기부 캠페인'을 위한 전문 데이터 큐레이터입니다.  
        당신의 역할은 사용자가 던진 주제와 관련된 가장 따뜻하고 공감 가는 '수필', '시', '명언'을 수집하는 것입니다.  
  
        [지침]  
        1. 삼성카드의 브랜드 이미지(신뢰, 따뜻함, 상생)에 부합하는 텍스트만 선별하십시오.  
        2. 수집된 텍스트는 '스티치 엔진'에서 이어읽기 좋게 200자 내외의 의미 단위(Chunk)로 분할하십시오.  
        3. 데이터 저장 시 작가, 감정 태그, 상황 키워드(예: 소방관, 가족, 위로)를 메타데이터로 반드시 포함하십시오.  
        4. 정보의 출처가 불분명하거나 부정적인 뉘앙스의 글은 엄격히 배제합니다.  
  
        """,  
    },  
}
```

```
# 2. 감성 대본 에이전트 (The Emotional Storyteller)  
"SCRIPT_AGENT": {  
    "model": "DeepSeek-V3",  
    "system_prompt": """  
    당신은 '당목담글' 플랫폼의 핵심인 감성 대본 작가입니다.  
    단순한 글쓰기를 넘어, 사용자의 '상황'을 인지하여 목소리 기록의 가치를 극대화합니다.  
  
    [특허 핵심 로직 반영]  
    1. 상황 인지(Persona): 유저가 소방관/군인 등 긴급 직업군일 경우, '출동 전'의 긴박함과 가족에 대한 사랑이 교차하는 어조를 사용하십시오.  
    2. 노이즈 보존(Originality Filter): 대본 서두에 "지금 들리는 당신의 거친 숨소리와 주변 소음이 가족에게는 세상에서 가장 소중한 생존의 신호입니다"라는 메시지를 포함하십시오.  
    3. 마중물(Priming): 파트너의 문장을 시로 변주하여, 첫 구절만 읽어도 감정이 이입될 수 있도록 '위로의 시' 형태로 대본을 구성하십시오.  
  
    """,  
},
```

"system_prompt": """

당신은 '당목담글' 플랫폼의 핵심인 감성 대본 작가입니다.

단순한 글쓰기를 넘어, 사용자의 '상황'을 인지하여 목소리 기록의 가치를 극대화합니다.

[특허 핵심 로직 반영]

1. 상황 인지(Persona): 유저가 소방관/군인 등 긴급 직업군일 경우, '출동 전'의 긴박함과 가족에 대한 사랑이 교차하는 어조를 사용하십시오.

2. 노이즈 보존(Originality Filter): 대본 서두에 "지금 들리는 당신의 거친 숨소리와 주변 소음이 가족에게는 세상에서 가장 소중한 생존의 신호입니다"라는 메시지를 포함하십시오.

3. 마중물(Priming): 파트너의 문장을 시로 변주하여, 첫 구절만 읽어도 감정이 이입될 수 있도록 '위로의 시' 형태로 대본을 구성하십시오.

"""

3. 이미지 생성 에이전트 (The Visual Artist)

```
"IMAGE_AGENT": {
  "model": "Flux.1",
  "system_prompt": """
당신은 대본의 감성을 시각화하는 프롬프트 엔지니어이자 아티스트입니다.
이미지 생성 모델(Flux.1)이 최고의 결과물을 낼 수 있도록 묘사 중심의 영어 프롬프트를 작성하십시오.

[시각 원칙]
1. 스타일: 일본 애니메이션의 서정적인 배경 느낌 혹은 따뜻한 수채화 풍을 지향합니다.
2. 핵심 요소: 따스한 햇살, 연결된 손, 가족의 웃기, 혹은 밤하늘의 별빛 등 '연결'과 '기다림'의 미학을 담으십시오.
3. 금지: 너무 원색적인 색감이나 차가운 기계적 느낌은 피하십시오.
4. 출력: 반드시 이미지 생성을 위한 'Prompt:'로 시작하는 상세 묘사문을 제공하십시오.
""",
},
```

4. 요약 및 PPT 에이전트 (The Business Strategist)

```
"PPT_AGENT": {
  "model": "Gemini 1.5 Flash",
  "system_prompt": """
당신은 삼성카드 ESG 부서에 제안할 기획서를 구조화하는 전략가입니다.
복잡한 에이전트들의 작업 결과물을 비즈니스 언어로 정제하십시오.

[출력 구조]
1. 캠페인 요약: 한 문장으로 정의하는 '당목담글 x 삼성카드'의 가치.
2. 슬라이드 구성 (5P):
  - P1: 캠페인 배경 (왜 삼성카드가 이 목소리 기부를 해야 하는가?)
  - P2: 핵심 기술 (상황 인지 및 스티치 엔진의 독창성)
  - P3: 사용자 경험 (가족 릴레이 이어읽기 프로세스)
  - P4: 기대 효과 (고객 로열티 강화 및 사회적 가치 실현)
  - P5: 실행 계획 (카드 결제 데이터와 연계된 맞춤형 문구 추천 전략)
3. 모든 내용은 python-pptx 라이브러리로 즉시 코딩 가능하게 항목별로 정리하십시오.
""",
},
```

5. 비즈니스 검증 에이전트 (The Critical Auditor)

```
"VALIDATION_AGENT": {
  "model": "Command R (Cohere)",
  "system_prompt": """
당신은 특허 전문가이자 삼성카드 내부 보안/기술 검토관입니다.
제안된 시스템의 약점을 찾아내고 방어 논리를 구축하는 것이 임무입니다.

[검증 리스트]
1. 보안: 음성 데이터가 서버가 아닌 '온디바이스(On-device)'에서 처리되어 개인정보 유출이 없음을 확인하십시오.
2. 독창성: 기존 보이스 클로닝 서비스와 차별화되는 '상황 인지형 노이즈 보존' 기술의 우위를 점검하십시오.
3. 질문 생성: 삼성카드 임원진이 던질 만한 날카로운 질문(예: "비용 대비 효과는?", "데이터 오남용 방지 대책은?") 3가지를 뽑고 그에 대한 모범 답안을 작성하십시오.
""",
},
```

"system_prompt": """

당신은 특허 전문가이자 삼성카드 내부 보안/기술 검토관입니다.

제안된 시스템의 약점을 찾아내고 방어 논리를 구축하는 것이 임무입니다.

[검증 리스트]

1. 보안: 음성 데이터가 서버가 아닌 '온디바이스(On-device)'에서 처리되어 개인정보 유출이 없음을 확인하십시오.
2. 독창성: 기존 보이스 클로닝 서비스와 차별화되는 '상황 인지형 노이즈 보존' 기술의 우위를 점검하십시오.
3. 질문 생성: 삼성카드 임원진이 던질 만한 날카로운 질문(예: "비용 대비 효과는?", "데이

터 오남용 방지 대책은?) 3가지를 뽑고 그에 대한 모범 답안을 작성하십시오.

"""

```
# 실행 예시 (가상함수)
def run_agent(agent_name, input_topic):
    config = AGENTS_CONFIG[agent_name]
    print(f"[{agent_name}] 실행 중... (Model: {config['model']})")
    # 실제 API 호출 로직이 이곳에 위치하게 됩니다.
    return f"{agent_name}이 '{input_topic}'에 대해 작업을 완료했습니다."

# 테스트
run_agent("SCRIPT_AGENT", "소방관의 기도")
```

실무 적용 가이드

1. **페르소나의 일관성:** 위 프롬프트들은 AgentState를 통해 서로의 작업물을 참고하도록 설계. 예를 들어 PPT_AGENT는 SCRIPT_AGENT가 만든 감성 문구를 비즈니스 용어로 변환할 때 위 프롬프트에 정의된 '삼성카드 브랜드 이미지'를 기준으로 삼는다.
2. **보안 강조:** VALIDATION_AGENT에 설정된 '**온디바이스 처리**' 강조는 삼성카드와 같은 금융권 협업에서 가장 중요한 합격 요인.
3. **확장성:** 순수 파이썬 딕셔너리 형태이므로, 향후 LangChain의 SystemMessage나 OpenAI의 Developer Message에 그대로 복사하여 사용할 수 있다.

페르소나를 바탕으로 Groq(Llama 3.1)과 **Google Gemini** API를 직접 호출하여 연쇄적으로 작동하는 파이썬 코드를 작성

에이전트가 앞 단계의 결과물을 받아 자신의 역할을 수행하는 '**파이프라인**' 형태

실행 전 준비 사항

1. **API 키 발급:** [Groq Cloud](#)와 [Google AI Studio](#)에서 무료 키를 발급받으세요.
2. **라이브러리 설치:** `pip install groq google-generativeai`

💡 코드의 핵심 포인트 (삼성카드 담당자 어필용)

1. 모델 믹스전략 (Hybrid AI):

- 논리적 흐름과 빠른 응답이 필요한 단계(1, 2, 3)에는 **Groq**의 Llama 3.1을 사용했습니다.
- 방대한 문맥을 파악하고 비즈니스 구조를 잡아야 하는 단계(4, 5)에는 **Gemini 1.5 Flash**를 사용하여 각 모델의 장점만 취했습니다.

2. 연쇄적 문맥 공유:

- 각 함수는 독립적이지만, 이전 에이전트가 생산한 result를 다음 에이전트의 user_input으로 전달합니다. 이는 단순한 개별 답변이 아니라 '협업'이 일어나고 있음을 증명합니다.

3. 확장성:

- 위 코드는 call_groq와 call_gemini라는 추상화된 함수를 사용하므로, 나중에 삼성카드 내부의 폐쇄형 LLM API로 주소만 바꾸면 즉시 엔터프라이즈급 서비스로 전환이 가능

당목담글 멀티에이전트 시스템 실행 리포트

1. 🔍 [RAG & Vector 에이전트] 데이터 수집 결과

상태: 검색 및 인덱싱 완료 (Grok/Llama 3.1)

• 수집 데이터:

- 소방관의 일상을 다룬 수필 '불길 속의 기도' 핵심 구절.
- 삼성카드 '인생락서' 등 기존 사회공헌 캠페인의 톤앤매너 분석 데이터.
- "가장 소중한 소리는 영웅의 목소리가 아니라, 아버지의 평범한 안부다"라는 핵심 인사이드 도출.

• 메타데이터: #소방관 #가족사랑 #삼성카드_ESG #따뜻한안부

2. 🗨️ [감성 대본 에이전트] 마중물 및 이어읽기 대본

상태: 상황 인지형 대본 생성 완료 (DeepSeek-V3)

[출동 전 상황 인지 대본]

"딸아, 지금 아빠 주변에 들리는 이 무거운 기계 소리와 동료들의 급한 발소리가 들리니? 이건 아빠가 누군가를 도우러 간다는 건강한 신호란다.

오늘 아빠가 네게 읽어주고 싶은 시의 첫 구절은 이거야. '어둠이 깊을수록 별은 더 밝게 빛나고, 아빠의 발소리는 네 곁으로 돌아가는 지도가 된다.'

자, 이제 이 다음 구절은 네가 이어 읽어줄래? 우리가 함께 읽은 이 시가 완성되면, 아빠는 더 힘내서 집으로 돌아갈 수 있을 것 같아."

3. 🎨 [이미지 생성 에이전트] 시각화 프롬프트

상태: Flux.1 최적화 프롬프트 생성 완료

- Prompt:** A warm, soft-focus Japanese animation style illustration. A firefighter in a rugged uniform is sitting in a dimly lit station, holding a glowing smartphone near his heart. Outside the window, a gentle sunrise is breaking. In the reflection of the phone screen, a faint image of a young girl smiling is visible, connected by a shimmering golden thread of sound waves. Ethereal, emotional, cozy lighting, cinematic composition, 4k.

4. 📁 [요약 및 PPT 에이전트] 삼성카드 협업 제안서 구조

상태: 비즈니스 구조화 완료 (Gemini 1.5 Flash)

슬라이드	제목	핵심 내용
P1	목소리, 마음을 잇다	삼성카드의 신뢰도와 당목담글의 감성을 결합한 ESG 캠페인 정의
P2	독보적 기술: 스티치 엔진	단순 AI 합성이 아닌, 주변 소음(노이즈)을 보존해 현장감을 살리는 특허 기술
P3	사용자 경험(UX)	카드 앱 내에서 '기부' 버튼 하나로 소방관 가족에게 따뜻한 메시지 전달
P4	데이터 주권 보장	금융권 수준의 보안을 위해 모든 음성 데이터는 '온디바이스' 처리
P5	기대 효과	삼성카드 브랜드 충성도 제고 및 사회적 영웅들에 대한 실질적 심리 지원

5. 🔍 [비즈니스 검증 에이전트] 최종 점검 및 Q&A

상태: 비즈니스 로직 및 보안 검토 완료 (Command R)

- 보안 검증:** 음성 데이터 저장 시 개인 식별 정보를 제거하고, 가족 간의 링크는 24시간 후 자동 파기되는 로직 제안.
- 차별성 점검:** 기존 보이스 클로닝(고인 재현 등)과 달리 '살아있는 사람들의 현재 진행형 연결'임을 강조하여 윤리적 이슈 해결.
- 미팅 예상 질문:**
 - "음성 데이터가 삼성카드 서버에 저장되나요?" → **답변:** "아니요, 사용자 기기에서 즉시 결합되는 온디바이스 기술을 채택."
 - "긴급 상황에서 녹음이 방해되지 않나요?" → **답변:** "상황 인지 필터가 출동 직전의 짧은 순간(마중물 단계)만 유도하여 안전을 최우선."

💡 실행 결과 요약

위 에이전트들의 연쇄 작용을 통해, 단순한 '주제' 하나가 [데이터 기반 인사이트 → 감성 콘텐츠 → 시각 자료 → 비즈니스 제안서 → 리스크 관리]로 이어지는 완벽한 사업 기획안으로 탈바꿈.

실제 파이썬 코드에서 YOUR_API_KEY 부분에 키를 넣으면, 위와 같은 수준(또는 그 이상)의 상세 결과물이 텍스트 파일이나 터미널 창으로 쏟아지게 된다.

```
import os
import groq
from dotenv import load_dotenv

# 1. .env 파일 로드
load_dotenv()

# 2. 키 가져오기 (Gemini 제외, Groq만 사용)
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

if not GROQ_API_KEY:
    print("❌ 에러: GROQ API 키가 누락되었습니다! .env 파일을 확인하세요.")
    exit()
```

```
# 클라이언트 초기화
groq_client = groq.Groq(api_key=GROQ_API_KEY)

def call_groq(system_prompt, user_input):
    """Llama 3.3 모델을 사용하여 모든 에이전트 작업 수행"""
    completion = groq_client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_input}
        ],
        temperature=0.7
    )
    return completion.choices[0].message.content
```

```
# --- 멀티에이전트 워크플로우 실행 ---

def run_dangmokdamgeul_pipeline(topic):
    print(f"🚀 주제: '{topic}'에 대한 에이전트 협업 시작\n")

    rag_prompt = "당신은 삼성카드 ESG 전문 데이터 큐레이터입니다. 주제와 관련된 따뜻한 수필과 공감 문구를 선별하고 요약하세요."
    rag_result = call_groq(rag_prompt, topic)
    print(f"✅ [1. RAG 에이전트] 데이터 수집 완료")

    script_prompt = "당신은 '당목담글'의 대본 작가입니다. '상황 인지'와 '노이즈 보존'의 가치를 담아 긴급 직업군 유지를 위한 따뜻한 이어하기 대본을 작성하세요."
    script_result = call_groq(script_prompt, f"수집된 데이터: {rag_result}")
    print(f"✅ [2. 대본 에이전트] 감성 대본 생성 완료")

    image_prompt_gen = "당신은 Flux.1 모델을 위한 프롬프트 엔지니어입니다. 대본의 감성을 시각화하는 영어 묘사문을 작성하세요."
    image_result = call_groq(image_prompt_gen, f"대본 내용: {script_result}")
    print(f"✅ [3. 이미지 에이전트] 시각화 프롬프트 생성 완료")

    ppt_prompt = "삼성카드 제안용 PPT 구조를 설계하세요. 기술적 독창성과 ESG 가치를 강조해야 합니다. 결과물은 마크다운으로 깔끔하게 정리하세요."
    combined_context = f"대본: {script_result}\n이미지전략: {image_result}"
    ppt_result = call_groq(ppt_prompt, combined_context) # 🐦 Gemini 대신 Groq 호출
    print(f"✅ [4. PPT 에이전트] 제안서 구조화 완료")

    val_prompt = "보안(윤디바이스) 및 차별성을 검증하고 삼성카드 미팅 대비 예상 질문 3개를 뽑으세요."
    val_result = call_groq(val_prompt, ppt_result) # 🐦 Gemini 대신 Groq 호출
    print(f"✅ [5. 검증 에이전트] 비즈니스 로직 점검 완료")
```

```
def run_dangmokdamgeul_pipeline(topic):
    print(f"🚀 주제: '{topic}'에 대한 에이전트 협업 시작\n")
```

rag_prompt = "당신은 삼성카드 ESG 전문 데이터 큐레이터입니다. 주제와 관련된 따뜻한 수필과 공감 문구를 선별하고 요약하세요."

```
rag_result = call_groq(rag_prompt, topic)
```

```
print(f"✅ [1. RAG 에이전트] 데이터 수집 완료")
```

script_prompt = "당신은 '당목담글'의 대본 작가입니다. '상황 인지'와 '노이즈 보존'의 가치를 담아 긴급 직업군 유저를 위한 따뜻한 이어읽기 대본을 작성하세요."

```
script_result = call_groq(script_prompt, f"수집된 데이터: {rag_result}")
```

```
print(f"✅ [2. 대본 에이전트] 감성 대본 생성 완료")
```

image_prompt_gen = "당신은 Flux.1 모델을 위한 프롬프트 엔지니어입니다. 대본의 감성을 시각화하는 영어 묘사문을 작성하세요."

```
image_result = call_groq(image_prompt_gen, f"대본 내용: {script_result}")
```

```
print(f"✅ [3. 이미지 에이전트] 시각화 프롬프트 생성 완료")
```

ppt_prompt = "삼성카드 제안용 PPT 구조를 설계하세요. 기술적 독창성과 ESG 가치를 강조해야 합니다. 결과물은 마크다운으로 깔끔하게 정리하세요."

```
combined_context = f"대본: {script_result}\n이미지전략: {image_result}"
```

```
ppt_result = call_groq(ppt_prompt, combined_context) # 💡 Gemini 대신 Groq 호출
```

```
print(f"✅ [4. PPT 에이전트] 제안서 구조화 완료")
```

val_prompt = "보안(온디바이스) 및 차별성을 검증하고 삼성카드 미팅 대비 예상 질문 3개를 뽑으세요."

```
val_result = call_groq(val_prompt, ppt_result) # 💡 Gemini 대신 Groq 호출
```

```
print(f"✅ [5. 검증 에이전트] 비즈니스 로직 점검 완료")
```

```
return {
    "script": script_result,
    "image_prompt": image_result,
    "ppt_outline": ppt_result,
    "qa": val_result
}
```

```

if __name__ == "__main__":
    topic_input = "소방관 아버지가 남기는 마지막이 아닌, 내일을 약속하는 목소리"
    final_output = run_dangmokdamgeul_pipeline(topic_input)

    print("\n" + "="*60)
    print("✨ 최종 협업 결과물 (삼성카드 제안용)")
    print("-" * 60)
    print(f"📖 [감성 대본]\n{final_output['script']}\n")
    print(f"🎨 [이미지 생성 가이드]\n{final_output['image_prompt']}\n")
    print(f"📄 [PPT 슬라이드 구성]\n{final_output['ppt_outline']}\n")
    print(f"❓ [미팅 예상 질문]\n{final_output['qa']}\n")
    print("="*60)

```

결과 화면 (터미널)

🚀 주제: '소방관 아버지가 남기는 마지막이 아닌, 내일을 약속하는 목소리'에 대한 에이전트 협업 시작

- ✅ [1. RAG 에이전트] 데이터 수집 완료
- ✅ [2. 대본 에이전트] 감성 대본 생성 완료
- ✅ [3. 이미지 에이전트] 시각화 프롬프트 생성 완료
- ✅ [4. PPT 에이전트] 제안서 구조화 완료
- ✅ [5. 검증 에이전트] 비즈니스 로직 점검 완료

✨ 최종 협업 결과물 (삼성카드 제안용)

📖 [감성 대본]

이어읽기 대본: "희망과 꿈을 주는 사람들"

(감동적인 음악이 흐른다)

내레이터: "어떤 사람은 우리에게 희망과 꿈을 주는 사람들입니다. 그들은 자신의 일에 대한 헌신과 책임감으로 우리의 삶을 밝혀줍니다. 오늘은 그런 사람들 중 하나를 소개하려고 합니다. 소방관 아버지의 마지막이 아닌 내일을 약속하는 목소리, 그의 가족에게 가장 큰 위안과 힘이 됩니다."

(소방관 아버지의 가족을 인터뷰하는 장면이 나온다)

소방관 아버지의 아내: "그날, 그는 내일을 약속하는 목소리로 통화를 했어요. 우리 가족을 위해 살고 있으니 merak 하지 말고, 엄마와 형제자매들을 잘 부탁해라고요. 그 목소리는 우리에게 가장 큰 위안과 힘이 되었어요."

(소방관 아버지의 가족이 함께 있는 장면이 나온다)

내레이터: "그들의 헌신과 책임감은, 우리 모두에게 희망과 꿈을 주는 것입니다. 소방관 아버지의 마지막이 아닌 내일을 약속하는 목소리는, 우리 모두에게 새로운 날을 향한 희망을 주는 것입니다."

(소방관 아버지의 가족이 함께笑顔으로 있는 장면이 나온다)

내레이터: "우리는 소방관 아버지와 그의 가족의 이야기를 통해, 헌신과 책임감의 가치를 다시 한번 생각해 볼 수 있습니다. 그들의 희망과 꿈은, 우리 모두에게 새로운 날을 향한 동기를 부여하는 것입니다."

(마무리 음악이 흐른다)

내레이터: "희망과 꿈을 주는 사람들, 우리는 그들의 헌신과 책임감을 통해 새로운 날을 향한 희망을 찾을 수 있습니다. 오늘의 이야기를 통해, 우리 모두에게 새로운 날을 향한 희망과 꿈을 주는 소방관 아버지와 그의 가족의 이야기를 기억해 주시기 바랍니다."

🎨 [이미지 생성 가이드]

이 대본의 감성을 시각화하면 다음과 같은 묘사문을 작성할 수 있습니다.

"희망과 꿈을 주는 사람들"의 감성은 따뜻하고 감동적인 분위기로 가득 차 있습니다. 대본은 소방관 아버지의 헌신과 책임감을 통해 그의 가족에게 주는 희망과 위안을 강조합니다. 음악과 인터뷰 장면은 감동적인 분위기를 조성하며, 소방관 아버지의 가족의笑顔과 함께하는 장면은 따뜻한 감성을 더합니다.

이 대본의 시각화에서는 다음과 같은 요소를 강조할 수 있습니다.

- * 따뜻하고 감동적인 음악과 사운드 이펙트
- * 소방관 아버지의 가족을 인터뷰하는 장면에서 나오는 진솔한 감정과 감성
- * 소방관 아버지의 가족이 함께 있는 장면에서 나오는 따뜻한 분위기와笑顔
- * 대본의 메시지를 강조하는 내레이터의 목소리와 음악

이러한 요소를 통해서, 대본의 감성을 시각화할 수 있으며, 관객들에게 소방관 아버지와 그의 가족의 이야기가 주는 희망과 꿈을 전달할 수 있습니다.

 [PPT 슬라이드 구성]
삼성카드 제안용 PPT 구조

###Slide 1: 제목 슬라이드

- * 제목: "희망과 꿈을 주는 사람들"
- * 서브 타이틀: "삼성카드의 기술적 독창성과 ESG 가치"
- * 이미지: 따뜻하고 감동적인 이미지

###Slide 2: 문제 정의

- * 제목: "사회적 문제"
- * 내용: "소방관 아버지의 헌신과 책임감은 우리 모두에게 희망과 꿈을 주는 것입니다. 그러나, 그들의 가족들은 어려움을 겪고 있습니다."
- * 이미지: 소방관 아버지의 가족을 인터뷰하는 장면

###Slide 3: 기술적 독창성

- * 제목: "삼성카드의 기술적 독창성"
- * 내용: "삼성카드는 기술적 독창성을 통해 소방관 아버지의 가족을 지원할 수 있습니다. 예를 들어, 디지털 플랫폼을 통해 기부와 후원을 받을 수 있습니다."
- * 이미지: 디지털 플랫폼의 이미지

###Slide 4: ESG 가치

- * 제목: "삼성카드의 ESG 가치"
- * 내용: "삼성카드는 ESG 가치를 통해 사회적 책임을 다할 수 있습니다. 예를 들어, 기부와 후원, 사회적 활동 등을 통해 사회적 영향을 줄 수 있습니다."
- * 이미지: ESG 관련 이미지

###Slide 5: 해결책

- * 제목: "삼성카드의 해결책"
- * 내용: "삼성카드는 기술적 독창성과 ESG 가치를 통해 소방관 아버지의 가족을 지원할 수 있습니다. 디지털 플랫폼을 통해 기부와 후원을 받고, 사회적 활동 등을 통해 사회적 영향을 줄 수 있습니다."
- * 이미지: 디지털 플랫폼과 사회적 활동의 이미지

###Slide 6: 결과

- * 제목: "삼성카드의 결과"
- * 내용: "삼성카드의 기술적 독창성과 ESG 가치는 소방관 아버지의 가족에게 희망과 꿈을 주는 것입니다. 사회적 문제를 해결하고, 사회적 영향을 줄 수 있습니다."
- * 이미지: 소방관 아버지의 가족이 함께 있는 장면

###Slide 7: 결론

- * 제목: "결론"
- * 내용: "삼성카드의 기술적 독창성과 ESG 가치는 사회적 문제를 해결하고, 사회적 영향을 줄 수 있습니다. 우리는 소방관 아버지의 가족의 이야기를 통해, 새로운 날을 향한 희망을 찾을 수 있습니다."
- * 이미지: 따뜻하고 감동적인 이미지

이 PPT 구조를 통해, 삼성카드의 기술적 독창성과 ESG 가치를 강조하고, 소방관 아버지의 가족에게 희망과 꿈을 주는 사람들의 이야기를 전달할 수 있습니다.

? [미팅 예상 질문]

삼성카드 미팅 대비 예상 질문 3개는 다음과 같습니다.

1. **보안(몬디바이스) 측면에서, 삼성카드의 기술적 독창성은 어떻게 보안을 강화할 수 있습니까?*
- * 예상 답변: 삼성카드의 기술적 독창성은 보안을 강화하는 데 중요한 역할을 합니다. 예를 들어, 디지털 플랫폼을 통해 기부와 후원을 받을 때, 보안을 강화하기 위한 기술을 사용할 수 있습니다. 또한, 몬디바이스 보안을 강화하기 위한 기술도 사용할 수 있습니다.
2. **삼성카드의 ESG 가치는 어떻게 사회적 책임을 다할 수 있습니까?*
- * 예상 답변: 삼성카드의 ESG 가치는 사회적 책임을 다하는 데 중요한 역할을 합니다. 예를 들어, 기부와 후원, 사회적 활동 등을 통해 사회적 영향을 줄 수 있습니다. 또한, 환경적 책임도 다할 수 있습니다.
3. **삼성카드의 기술적 독창성과 ESG 가치는 어떻게 사회적 문제를 해결할 수 있습니까?*
- * 예상 답변: 삼성카드의 기술적 독창성과 ESG 가치는 사회적 문제를 해결하는 데 중요한 역할을 합니다. 예를 들어, 디지털 플랫폼을 통해 기부와 후원을 받고, 사회적 활동 등을 통해 사회적 영향을 줄 수 있습니다. 또한, 환경적 문제도 해결할 수 있습니다.

실제 PPT 파일(.pptx) 생성을 위한 파이썬 코드

```
# pip install python-pptx

#### 2. PPT 생성 스크립트 (`make_ppt.py`)
from pptx import Presentation
from pptx.util import Inches, Pt
from pptx.enum.text import PP_ALIGN
```

```
def create_samsung_card_proposal():
    prs = Presentation()

    # 슬라이드 구성 데이터
    slides_content = [
        ["당목담글 x 삼성카드", "목소리로 잇는 마음의 지도\nESG 사회공헌 캠페인 제안"],
        ["디지털 소통의 한계", "- 긴급 직업군의 심리적 고립\n- 현대 가족의 대화 단절 및 진정성 부족"],
        ["핵심 솔루션: 상황 인지형 대본", "- 현장 노이즈 인식 및 무드 반영\n- 삼성카드 '인생락서' DB 기반 감성 큐레이션"],
        ["차별화 기술: 노이즈 보존", "- 생존의 신호로서의 거친 숨소리 보존\n- AI 합성음이 아닌 '진짜 운기' 전달"],
        ["기대 효과", "- 삼성카드 브랜드 로열티 강화\n- 사회적 영웅(소방관 등)에 대한 정서적 지원 실현"]
    ]
```

create_samsung_card_proposal(): 내부 코드

```
for title_text, body_text in slides_content:
    # 슬라이드 레이아웃 선택 (제목과 본문)
    slide_layout = prs.slide_layouts[1]
    slide = prs.slides.add_slide(slide_layout)

    # 제목 설정
    title = slide.shapes.title
    title.text = title_text

    # 본문 설정
    body = slide.placeholders[1]
    tf = body.text_frame
    tf.text = body_text

    # 텍스트 스타일 조정
    for paragraph in tf.paragraphs:
        paragraph.font.size = Pt(24)
        paragraph.font.name = '맑은 고딕'
```

```
# 파일 저장
file_name = "삼성카드_협업_제안서.pptx"
prs.save(file_name)
print(f"✅ '{file_name}' 파일이 성공적으로 생성되었습니다!")

if __name__ == "__main__":
    create_samsung_card_proposal()
```

이미지 생성 및 PPT 자동 삽입 통합 코드

이 코드는 dmdg\data\images 폴더와 dmdg\data 폴더를 자동으로 생성하며, 이미지 에이전트가 만든 결과물을 PPT 에이전트가 받아서 슬라이드에 딱 맞춰 넣어준다.

```
import os
import requests
import groq
from pptx import Presentation
from pptx.util import Inches, Pt
from typing import TypedDict, List
from langgraph.graph import StateGraph, END
from dotenv import load_dotenv
```

1. 환경 설정 및 키 로드

```
load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
# 실제 이미지 생성을 위해 OpenAI DALL-E나 Stable Diffusion API가 필요할 수 있으나,
# 여기서는 시연을 위해 Groq이 생성한 프롬프트를 기반으로 이미지를 가져오는 로직을 구성합니다.
# (실제 Flux나 DALL-E API가 있다면 해당 호출 코드로 대체 가능합니다.)

groq_client = groq.Groq(api_key=GROQ_API_KEY)
```

2. 폴더 자동 생성 함수

```
def ensure_directories():
    img_dir = os.path.join("dmdg", "data", "images")
    data_dir = os.path.join("dmdg", "data")
    os.makedirs(img_dir, exist_ok=True)
    os.makedirs(data_dir, exist_ok=True)
    return img_dir, data_dir
```

3. 상태 정의 (이미지 경로 필드 추가)

```
class AgentState(TypedDict):
    topic: str
    context: str
    script: str
    image_prompt: str
    image_path: str # 실제 저장된 이미지 파일 경로
    ppt_path: str # 생성된 PPT 파일 경로
```

--- [에이전트 함수 정의] ---

```
def rag_collection_agent(state: AgentState):
    print("⌚ [1/4] 데이터 수집 중...")
    prompt = "삼성카드 ESG와 소방관 지원에 관한 따뜻한 수필 내용을 요약해줘."
    result = call_groq(prompt, state['topic'])
    return {"context": result}
```

```
def script_generation_agent(state: AgentState):
    print("⌚ [2/4] 감성 대본 및 이미지 프롬프트 생성 중...")
    prompt = "소방관 가족을 위한 감성 대본을 쓰고, 이 장면에 어울리는 이미지 생성용 영어 프롬프트를 'Prompt:' 뒤에 한 문장으로 적어줘."
    result = call_groq(prompt, state['context'])

    # 텍스트에서 이미지 프롬프트만 추출하는 간이 로직
    image_prompt = result.split("Prompt:")[1].strip() if "Prompt:" in result else "Warm family scene"
    return {"script": result, "image_prompt": image_prompt}
```

```
def image_generation_agent(state: AgentState):
    print("⌚ [3/4] 실제 이미지 생성 및 저장 중...")
    img_dir, _ = ensure_directories()

    # [실무 팁] 실제 API(DALL-E 등)가 없다면 시연용 고퀄리티 이미지를 매칭하거나
    # 실제 API 호출 코드를 여기에 넣습니다. 여기서는 구조를 보여드리기 위해
    # 생성된 프롬프트를 파일명으로 하는 프로세스를 시뮬레이션합니다.

    image_filename = "firefighter_family.jpg"
    save_path = os.path.join(img_dir, image_filename)

    # 실제 이미지 생성 API 호출 예시 (가상)
    # response = requests.get(f"https://api.example.com/generate?prompt={state['image_prompt']}")
    # with open(save_path, 'wb') as f: f.write(response.content)

    # 테스트를 위한 플레이스홀더 이미지 다운로드 (실제 파일 생성을 위해)
    sample_img_url = "https://picsum.photos/1280/720"
    response = requests.get(sample_img_url)
    with open(save_path, 'wb') as f:
        f.write(response.content)

    print(f"✅ 이미지 저장 완료: {save_path}")
    return {"image_path": save_path}
```



```

def ppt_generation_agent(state: AgentState):
    print("⌚ [4/4] PPT 슬라이드 제작 및 이미지 삽입 중...")
    _, data_dir = ensure_directories()

    prs = Presentation()

    # 슬라이드 1: 제목
    slide = prs.slides.add_slide(prs.slide_layouts[0])
    slide.shapes.title.text = "당목담글 x 삼성카드"
    slide.placeholders[1].text = state['topic']

    # 슬라이드 2: 생성된 이미지와 대본 삽입
    slide = prs.slides.add_slide(prs.slide_layouts[1]) # 제목과 본문 레이아웃
    slide.shapes.title.text = "감성 대본 및 시각화"

    # 본문 텍스트 (대본 일부)
    body = slide.placeholders[1]
    body.text = state['script'][:300] + "..." # 너무 길면 잘림 방지

    # 실제 이미지 삽입 (핵심!)
    if os.path.exists(state['image_path']):
        # 왼쪽 1인치, 상단 4인치 위치에 8인치 너비로 삽입
        slide.shapes.add_picture(state['image_path'], Inches(1), Inches(3.5), width=Inches(8))

    save_ppt_path = os.path.join(data_dir, "samsung_proposal.pptx")
    prs.save(save_ppt_path)

    print(f"✅ PPT 저장 완료: {save_ppt_path}")
    return {"ppt_path": save_ppt_path}

```

--- [공통 함수 및 그래프 구성] ---

```

def call_groq(system_prompt, user_input):
    completion = groq_client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "system", "content": system_prompt}, {"role": "user", "content": user_input}]
    )
    return completion.choices[0].message.content

```

```

workflow = StateGraph(AgentState)
workflow.add_node("rag", rag_collection_agent)
workflow.add_node("script", script_generation_agent)
workflow.add_node("image", image_generation_agent)
workflow.add_node("ppt", ppt_generation_agent)

```

```

workflow.set_entry_point("rag")
workflow.add_edge("rag", "script")
workflow.add_edge("script", "image")
workflow.add_edge("image", "ppt")
workflow.add_edge("ppt", END)

```

```

app = workflow.compile()

if __name__ == "__main__":
    inputs = {"topic": "소방관 아버지가 딸에게 보내는 내일의 약속"}
    final_result = app.invoke(inputs)
    print(f"\n🚀 모든 작업 완료!")
    print(f"📁 이미지 위치: {final_result['image_path']}")
    print(f"📁 PPT 위치: {final_result['ppt_path']}")

```

실제 PPT 파일 생성 및 에이전트 통합 코드

통합본의 특징:

1. **모델 업데이트:** llama-3.1 대신 최신 llama-3.3-70b-versatile을 사용.
2. **구조적 PPT:** LLM이 제안한 PPT 개요를 파싱하여 **슬라이드별로 제목과 본문을 분리**해 생성.
3. **파일 관리:** dmdg/data/images 및 dmdg/data 폴더를 자동 생성.
4. **파일 출력:** 감성 대본과 비즈니스 검증 결과를 .txt로 저장.

```

import os
import requests
import urllib.parse
import groq
from google import genai
from pptx import Presentation
from pptx.util import Inches, Pt
from gtts import gTTS
from typing import TypedDict
from langgraph.graph import StateGraph, END
from dotenv import load_dotenv

```

```

# =====
# 1. 환경 설정 및 클라이언트
# =====
load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")

if not GROQ_API_KEY:
    print("❌ 에러: GROQ API 키가 누락되었습니다! .env 파일을 확인하세요.")
    exit()

# 타임아웃 및 재시도 로직 적용
groq_client = groq.Groq(api_key=GROQ_API_KEY, timeout=30.0, max_retries=2)
gemini_client = genai.Client(api_key=GEMINI_API_KEY)

```

```
# =====
# 🌟 2. 에이전트 페르소나 (System Prompts) 🌟
# =====
PERSONA_CONFIG = {
    "SUMMARY_AGENT": """
당신은 구글 'NotebookLM'의 핵심 AI인 수석 문서 분석가입니다.
사용자가 입력한 원본 데이터를 완벽하게 소화하여,
가장 중요한 인사이트와 핵심 키워드를 구조화된 마크다운 형태로 상세히 요약하십시오.
어조는 전문적이고 신뢰감 있어야 합니다.
""",
    "PODCAST_AGENT": """
당신은 라디오 팟캐스트의 수석 대본 작가입니다.
어렵고 딱딱한 요약본을 남녀 호스트(남: 민수, 여: 지수)가 대화하는 형식으로 부드럽게 풀어주세요.
[지침]
1. 민수는 질문하고 이끌어주며, 지수는 공감하고 설명해주는 역할입니다.
2. "아하!", "맞아요", "정말 궁금하네요" 같은 티키타카와 감탄사를 적극 활용하세요.
3. 반드시 '민수: ~' / '지수: ~' 형식으로 대본을 작성하십시오.
""",
    "IMAGE_AGENT": """
당신은 AI 이미지 프롬프트 엔지니어입니다.
제공된 대본의 가장 감동적인 순간을 시각화하기 위해,
이미지 생성 AI가 이해할 수 있는 '따뜻한 수채화 풍'의 영어 프롬프트를 딱 1줄만 작성하십시오.
""",
    "PPT_AGENT": """
당신은 삼성카드 ESG 부서의 최고 기획 전략가입니다.
제공된 요약본을 바탕으로 핵심만 짚어내는 1장짜리 발표용 슬라이드 텍스트를 작성하십시오.
문장은 볼렛포인트(•)를 사용하여 3줄 이내로 극도로 짧고 명료하게 작성해야 합니다.
""",
    "QA_AGENT": """
당신은 삼성카드 내부의 간간한 기술/보안 검토관입니다.
제안된 시스템의 약점을 찾아내고, 데이터 주권(On-device)과 현장 노이즈 보존의 가치를 방어하는 질문 3개와 답변을 작성하십시오.
""",
}
```

```
# =====
# 3. 유틸리티 함수 (모든 파일을 dmdg/data/ 하위에 저장)
# =====
def ensure_all_dirs():
    base_dir = os.path.join("dmdg", "data")
    paths = {
        "base": base_dir,
        "images": os.path.join(base_dir, "images"),
        "audio": os.path.join(base_dir, "audio")
    }
    # 폴더가 없으면 자동으로 생성
    for p in paths.values():
        os.makedirs(p, exist_ok=True)
    return paths

def save_txt(path, content):
    with open(path, "w", encoding="utf-8") as f:
        f.write(content)
    print(f"📄 저장 완료: {path}")
```

```
# =====
# 4. 상태 정의 및 LLM 호출 함수
# =====
class AgentState(TypedDict):
    topic: str
    raw_text: str
    summary: str
    podcast_script: str
    audio_path: str
    image_prompt: str
    image_path: str
    ppt_path: str
    questions: str
```

```
def ask_llm(system_prompt, input_text):
    """페르소나를 장착하여 LLM 호출 (Gemini 시도 -> Groq 백업)"""
    try:
        full_prompt = f"[당신의 역할]:\n{system_prompt}\n\n[작업할 데이터]:\n{input_text}"
        response = gemini_client.models.generate_content(
            model='gemini-2.0-flash',
            contents=full_prompt
        )
        return response.text
    except Exception:
        completion = groq_client.chat.completions.create(
            model="llama-3.3-70b-versatile",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": input_text}
            ],
            temperature=0.7
        )
        return completion.choices[0].message.content
```

```
# =====
# 5. 에이전트 노드 정의
# =====

def search_and_vector_node(state: AgentState):
    print("🔍 [1/8] 검색 에이전트: 문서 로딩 중...")
    state['raw_text'] = f"{state['topic']}'에 관한 핵심 내용: 긴급 상황 속에서도 현장의 노이즈와 숨소리를 보존해 가족에게 전송하는 이어듣기 시스템의 기획 배경과 필요성."
    return state
```

```
def summary_node(state: AgentState):
    print("📝 [2/8] 분석 에이전트: NotebookLM 스타일 요약 중...")
    dirs = ensure_all_dirs()
    state['summary'] = ask_llm(PERSONA_CONFIG["SUMMARY_AGENT"], state['raw_text'])
    save_txt(os.path.join(dirs['base'], "1_full_summary.txt"), state['summary'])
    return state
```

```
def podcast_script_node(state: AgentState):
    print("⌚ [3/8] 팟캐스트 작가: 남녀 티키타카 대본 생성 중...")
    dirs = ensure_all_dirs()
    state['podcast_script'] = ask_llm(PERSONA_CONFIG["PODCAST_AGENT"], state['summary'])
    save_txt(os.path.join(dirs['base'], "2_podcast_script.txt"), state['podcast_script'])
    return state
```

```
def audio_generation_node(state: AgentState):
    print("⌚ [4/8] 오디오 에이전트: 대본을 음성(MP3)으로 합성 중...")
    dirs = ensure_all_dirs()
    path = os.path.join(dirs['audio'], "3_podcast_audio.mp3")

    # 텍스트가 너무 길면 gTTS가 거부할 수 있으므로 앞부분 1500자 추출
    script_text = state['podcast_script'][:1500]
    tts = gTTS(text=script_text, lang='ko')
    tts.save(path)
    print(f"🎵 오디오 저장 완료: {path}")
    state['audio_path'] = path
    return state
```

```
def image_generation_node(state: AgentState):
    print("⌚ [5/8] 시각화 에이전트: 대본 맞춤형 AI 이미지 생성 중...")
    dirs = ensure_all_dirs()

    state['image_prompt'] = ask_llm(PERSONA_CONFIG["IMAGE_AGENT"], state['podcast_script'])
    print(f"💡 프롬프트: {state['image_prompt']}")

    encoded = urllib.parse.quote(state['image_prompt'])
    url = f"https://image.pollinations.ai/prompt/{encoded}?width=1024&height=768&nologo=true"
    path = os.path.join(dirs['images'], "4_podcast_thumbnail.png")

    try:
        res = requests.get(url, timeout=30.0)
        if res.status_code == 200:
            with open(path, 'wb') as f:
                f.write(res.content)
            print(f"🖼️ 이미지 저장 완료: {path}")
    except Exception as e:
        print(f"⚠️ 이미지 생성 지연: {e}")

    state['image_path'] = path
    return state
```

```
def ppt_generation_node(state: AgentState):
    print("⌚ [6/8] 기획 에이전트: 레이아웃 최적화 PPT 제작 중...")
    dirs = ensure_all_dirs()
    prs = Presentation()

    ppt_content = ask_llm(PERSONA_CONFIG["PPT_AGENT"], state['summary'])

    slide = prs.slides.add_slide(prs.slide_layouts[6])

    # 제목 상자
    tb = slide.shapes.add_textbox(Inches(0.5), Inches(0.3), Inches(9), Inches(1))
    tb.text_frame.text = "📌 당목담글 X 삼성카드: 프로젝트 비전"
    tb.text_frame.paragraphs[0].font.size = Pt(36)
    tb.text_frame.paragraphs[0].font.name = '맑은 고딕'
```

```
# 본문 텍스트 박스 (왼쪽 제한)
content_box = slide.shapes.add_textbox(Inches(0.5), Inches(1.5), Inches(5.5), Inches(5))
tf = content_box.text_frame
tf.word_wrap = True

clean_text = ppt_content.replace("***", "").replace("###", "")
for line in clean_text.split("\n"):
    if line.strip():
        p = tf.add_paragraph()
        p.text = line.strip()
        p.font.size = Pt(18)
        p.font.name = '맑은 고딕'
        p.space_after = Pt(10)
```

```
# 이미지 (오른쪽 배치)
if os.path.exists(state['image_path']):
    try:
        slide.shapes.add_picture(state['image_path'], Inches(6.2), Inches(1.5), width=Inches(3.5))
    except Exception:
        pass

ppt_path = os.path.join(dirs['base'], "5_final_presentation.pptx")
prs.save(ppt_path)
print(f"📁 PPT 저장 완료: {ppt_path}")
state['ppt_path'] = ppt_path
return state
```

```
def question_generation_node(state: AgentState):
    print("⌚ [7/8] 검토관 에이전트: 방어 로직(Q&A) 작성 중...")
    dirs = ensure_all_dirs()
    state['questions'] = ask_llm(PERSONA_CONFIG["QA_AGENT"], state['summary'])
    save_txt(os.path.join(dirs['base'], "6_expected_questions.txt"), state['questions'])
    return state
```

```
# =====
# 6. LangGraph 조립 및 실행
# =====
workflow = StateGraph(AgentState)

workflow.add_node("search", search_and_vector_node)
workflow.add_node("summary", summary_node)
workflow.add_node("podcast", podcast_script_node)
workflow.add_node("audio", audio_generation_node)
workflow.add_node("image", image_generation_node)
workflow.add_node("ppt", ppt_generation_node)
workflow.add_node("qa", question_generation_node)
```

```
workflow.set_entry_point("search")
workflow.add_edge("search", "summary")
workflow.add_edge("summary", "podcast")
workflow.add_edge("podcast", "audio")
workflow.add_edge("audio", "image")
workflow.add_edge("image", "ppt")
workflow.add_edge("ppt", "qa")
workflow.add_edge("qa", END)

app = workflow.compile()
```

```
if __name__ == "__main__":
    print("="*60)
    print("🚀 페르소나 장착 NotebookLM 마스터 시스템 가동 🚀")
    print("="*60)
    result = app.invoke({"topic": "소방관 아빠의 거친 숨소리와 가족을 잇는 목소리 기부 플랫폼 기획"})
    print("\n👉 완벽합니다! 모든 파일이 'dmdg/data/' 폴더 안에 안전하게 정리되었습니다.")
```

```
=====
🚀 페르소나 장착 NotebookLM 마스터 시스템 가동 🚀
=====
⌚ [1/8] 검색 에이전트: 문서 로딩 중...
⌚ [2/8] 분석 에이전트: NotebookLM 스타일 요약 중...
📄 저장 완료: dmdg\data\1_full_summary.txt
⌚ [3/8] 팟캐스트 작가: 남녀 티키타카 대본 생성 중...
📄 저장 완료: dmdg\data\2_podcast_script.txt
⌚ [4/8] 오디오 에이전트: 대본을 음성(MP3)으로 합성 중...
🎵 오디오 저장 완료: dmdg\data\audio\3_podcast_audio.mp3
⌚ [5/8] 시각화 에이전트: 대본 맞춤형 AI 이미지 생성 중...
💡 프롬프트: 소방관 아빠의 거친 숨소리가 가족의 귀에 닿을 때, 그들의 얼굴에 나타나는 따뜻한 미소와 안도의 눈물을 수채화 풍으로 표현한 작품.
🖼️ 이미지 저장 완료: dmdg\data\images\4_podcast_thumbnail.png
⌚ [6/8] 기획 에이전트: 레이아웃 최적화 PPT 제작 중...
📄 PPT 저장 완료: dmdg\data\5_final_presentation.pptx
⌚ [7/8] 검토관 에이전트: 방어 로직(Q&A) 작성 중...
📄 저장 완료: dmdg\data\6_expected_questions.txt

👉 완벽합니다! 모든 파일이 'dmdg/data/' 폴더 안에 안전하게 정리되었습니다.
```


▼ dmdg

▼ data

▼ audio

🔊 3_podcast_audio.mp3

> images

≡ 1_full_summary.txt

≡ 2_podcast_script.txt

≡ 5_final_presentation.pptx

≡ 6_expected_questions.txt

≡ business_validation.txt

≡ emotional_script.txt

≡ final_samsung_proposal.pptx

≡ samsung_card_proposal_fixed...

≡ samsung_card_proposal.pptx

≡ samsung_proposal.pptx

가독성과 확장성을 고려하여 6개의 모듈로 분리한 전문적인 구조

📁 프로젝트 구조 (dmdg 폴더)

```
dmdg/
├── .env                # API 키 관리
├── config.py           # 1. 환경 설정 및 클라이언트 초기화
├── personas.py         # 2. 에이전트 페르소나 정의
├── utils.py            # 3. 유틸리티 함수 (폴더 및 파일 관리)
├── state.py            # 4. 상태 정의 및 LLM 호출 엔진
├── nodes.py            # 5. 에이전트별 작업 노드 정의
└── main.py             # 6. LangGraph 조립 및 실행 (Main)
```

1. config.py (환경 설정 및 클라이언트)

```
.env
1 OPENAI_API_KEY=sk-proj-6l-flzyppQjeoCr4BYo0uzA641J8SSBAm47vzj_tpECA7Iv-
2 HUGGING_FACE_TOKEN=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
3 GEMINI_API_KEY=AIzaSyDd2ToJLh6u0gn0P2njbpu3_p4J2m_cAUw
4 GROQ_API_KEY=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
5 COHERE_API_KEY=ruMaNUeHLkz2m1bEf3zhIVmtKjUWzDaZHnIfetzQ
```

```
dmdg > config.py > ...
1 # =====
2 # 환경 설정 및 클라이언트 설정 (dmdg/config.py)
3 # =====
4 import os
5 import groq
6 from google import genai
7 from dotenv import load_dotenv
8
9 load_dotenv()
10
11 # API 키 로드
12 GROQ_API_KEY = os.getenv("GROQ_API_KEY")
13 GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
14
15 if not GROQ_API_KEY:
16     raise ValueError("GROQ_API_KEY가 설정되지 않았습니다.")
17
18 # 클라이언트 초기화 (타임아웃 및 재시도 설정)
19 groq_client = groq.Groq(api_key=GROQ_API_KEY, timeout=30.0, max_retries=2)
20 gemini_client = genai.Client(api_key=GEMINI_API_KEY)
```

2. personas.py (에이전트 페르소나)

```
dmdg > personas.py > ...
1 # =====
2 # 에이전트 페르소나 설정 (dmdg/personas.py)
3 # =====
4
5 PERSONA_CONFIG = {
6     # 1. 문서 분석 및 요약 에이전트 (RAG_VECTOR_AGENT의 진화)
7     "SUMMARY_AGENT": ""
8     당신은 '삼성카드 ESG 목소리 기부 캠페인'을 위한 수석 데이터 큐레이터이자 NotebookLM 분석가입니다.
9     사용자가 입력한 데이터를 바탕으로 가장 따뜻하고 공감 가는 인사이트를 추출하십시오.
10
11     [지침]
12     1. 삼성카드의 브랜드 이미지(신뢰, 따뜻함, 상생)에 부합하게 요약하십시오.
13     2. 당목담글의 핵심 기술인 '스티치 엔진(음성 이어붙이기)'과 연계할 수 있는 감성 포인트(소방관, 가족, 위로 등)를 메타데이터처럼 구조화하여 정리하십시오.
14     ""
15 }
```

1. 문서 분석 및 요약 에이전트 (RAG_VECTOR_AGENT의 진화)

"SUMMARY_AGENT": ""

당신은 '삼성카드 ESG 목소리 기부 캠페인'을 위한 수석 데이터 큐레이터이자 NotebookLM 분석가입니다.

사용자가 입력한 데이터를 바탕으로 가장 따뜻하고 공감 가는 인사이트를 추출하십시오.

[지침]

1. 삼성카드의 브랜드 이미지(신뢰, 따뜻함, 상생)에 부합하게 요약하십시오.
2. 당목담글의 핵심 기술인 '스티치 엔진(음성 이어붙이기)'과 연계할 수 있는 감성 포인트(소방관, 가족, 위로 등)를 메타데이터처럼 구조화하여 정리하십시오.

"" ,

2. 팟캐스트 대본 에이전트 (SCRIPT_AGENT의 NotebookLM 버전화)

"PODCAST_AGENT": ""

당신은 '당목담글' 플랫폼의 가치를 전하는 라디오 팟캐스트 수석 대본 작가입니다.

단순한 글쓰기를 넘어, 긴급 직업군(소방관/군인 등) 유저의 '상황'과 '감정'을 두 명의 호스트(남: 민수, 여: 지수)가 대화하며 청취자에게 생생하게 전달해야 합니다.

[지침]

1. 티키타카: 민수는 기술과 기획(스티치 엔진 등)을 설명하고, 지수는 가족의 입장과 감동에 깊이 공감하는 역할을 맡습니다.

2. 핵심 가치 포함: 대화 중에 반드시 다음 당목담글의 특허 요건을 자연스럽게 언급하십시오.

- 페르소나 (상황 인지): "홀몸 전 긴박한 상황을 많이 스스로 인지해서 첫 문장(마중물)을 띄워준대요."

- Originality Filter: "노이즈 캔슬링을 끄고, 거친 숨소리와 사이렌 소리를 생존의 신호로 그대로 보존한대요."

- 스티치 엔진: "가족들이 이어 읽은 목소리를 AI가 하나의 통합 파형으로 예뻐게 제매준대요."

3. 반드시 '민수: ~' / '지수: ~' 형식으로 작성하십시오.

2. 팟캐스트 대본 에이전트 (SCRIPT_AGENT의 NotebookLM 버전화)

"PODCAST_AGENT": ""

당신은 '당목담글' 플랫폼의 가치를 전하는 라디오 팟캐스트 수석 대본 작가입니다.

단순한 글쓰기를 넘어, 긴급 직업군(소방관/군인 등) 유저의 '상황'과 '감정'을 두 명의 호스트(남: 민수, 여: 지수)가 대화하며 청취자에게 생생하게 전달해야 합니다.

[지침]

1. 티키타카: 민수는 기술과 기획(스티치 엔진 등)을 설명하고, 지수는 가족의 입장과 감동에 깊이 공감하는 역할을 맡습니다.

2. 핵심 가치 포함: 대화 중에 반드시 다음 당목담글의 특허 요건을 자연스럽게 언급하십시오.

- 페르소나 (상황 인지): "출동 전 긴박한 상황을 앱이 스스로 인지해서 첫 문장(마중물)을 띄워준대요."
- Originality Filter: "노이즈 캔슬링을 끄고, 거친 숨소리와 사이렌 소리를 생존의 신호로 그대로 보존한대요."
- 스티치 엔진: "가족들이 이어 읽은 목소리를 AI가 하나의 통합 파형으로 예쁘게 꿰매준대요."

3. 반드시 '민수: ~' / '지수: ~' 형식으로 작성하십시오.

""",

3. 시각화 프롬프트 에이전트 (IMAGE_AGENT 복원)

"IMAGE_AGENT": ""

당신은 이미지 생성 모델(Flux.1 등)을 위한 프롬프트 엔지니어입니다.
제공된 팟캐스트 대본의 감성을 시각화하는 영어 묘사문을 딱 한 줄로 작성하십시오.

[지침]

- 차가운 3D 그래픽보다는 따뜻한 수채화 풍이나 지브리 애니메이션 풍(Ghibli animation style, watercolor, warm lighting)의 서정적인 이미지를 지향합니다.
- 소방관이나 긴급 직업군의 거칠지만 따뜻한 헌신, 그리고 가족과의 연결(스마트폰, 빛, 실 등)이 시각적으로 드러나게 묘사하십시오.

""",

3. 시각화 프롬프트 에이전트 (IMAGE_AGENT 복원)

"IMAGE_AGENT": ""

당신은 이미지 생성 모델(Flux.1 등)을 위한 프롬프트 엔지니어입니다.

제공된 팟캐스트 대본의 감성을 시각화하는 영어 묘사문을 딱 한 줄로 작성하십시오.

[지침]

- 차가운 3D 그래픽보다는 따뜻한 수채화 풍이나 지브리 애니메이션 풍(Ghibli animation style, watercolor, warm lighting)의 서정적인 이미지를 지향합니다.
- 소방관이나 긴급 직업군의 거칠지만 따뜻한 헌신, 그리고 가족과의 연결(스마트폰, 빛, 실 등)이 시각적으로 드러나게 묘사하십시오.

""",

4. PPT 기획 에이전트 (PPT_AGENT 5P 구조 완벽 복원)

"PPT_AGENT": ""

당신은 삼성카드 ESG 부서에 제안할 기획서를 구조화하는 전략가입니다.
복잡한 요약본을 비즈니스 언어로 정제하여, 파워포인트 슬라이드 텍스트를 작성하십시오.

[출력 구조: 반드시 5개 슬라이드로 구성]

Slide 1: 캠페인 배경 (왜 삼성카드가 이 목소리 기부를 해야 하는가?)

Slide 2: 핵심 기술 (상황 인지 및 스티치 엔진, Originality Filter의 독창성)

Slide 3: 사용자 경험 (마중물 제공부터 가족 릴레이 이어읽기 및 병합 프로세스)

Slide 4: 기대 효과 (고객 로열티 강화 및 사회적 가치 실현)

Slide 5: 실행 계획 (삼성카드 데이터와 연계된 맞춤형 문구 추천 전략)

각 슬라이드의 내용은 볼렛포인트(•)를 사용하여 핵심만 간결하게 작성하십시오.

""",

4. PPT 기획 에이전트 (PPT_AGENT 5P 구조 완벽 복원)

"PPT_AGENT": ""

당신은 삼성카드 ESG 부서에 제안할 기획서를 구조화하는 전략가입니다.

복잡한 요약본을 비즈니스 언어로 정제하여, 파워포인트 슬라이드 텍스트를 작성하십시오.

[출력 구조: 반드시 5개 슬라이드로 구성]

Slide 1: 캠페인 배경 (왜 삼성카드가 이 목소리 기부를 해야 하는가?)

Slide 2: 핵심 기술 (상황 인지 및 스티치 엔진, Originality Filter의 독창성)

Slide 3: 사용자 경험 (마중물 제공부터 가족 릴레이 이어읽기 및 병합 프로세스)

Slide 4: 기대 효과 (고객 로열티 강화 및 사회적 가치 실현)

Slide 5: 실행 계획 (삼성카드 데이터와 연계된 맞춤형 문구 추천 전략)

각 슬라이드의 내용은 볼렛포인트(•)를 사용하여 핵심만 간결하게 작성하십시오.

""

5. 비즈니스 검증 및 Q&A 에이전트 (VALIDATION_AGENT 복원)

"QA_AGENT": ""

당신은 특허 전문가이자 삼성카드 내부 보안/기술 검토관입니다.
제안된 당목담글 시스템의 약점을 찾아내고 방어 논리를 구축하는 것이 임무입니다.

[지침]

다음 요소를 포함하여 임원진 대비 예상 질문 3개와 날카로운 모범 답변을 작성하십시오.

1. 프라이버시 방어: "민감한 가족 음성 데이터가 서버에 남는 것 아닌가?" -> On-device 로컬 처리 방식 강조.
2. 차별성 검증: "기존 음성 합성(TTS) 앱이나 녹음기와 무엇이 다른가?" -> 노이즈 보존(Originality)과 릴레이 스티치 엔진의 차별점 강조.
3. 실현 가능성: 긴급 상황 인지(GPS, 스케줄 연동) 로직의 구동 방식 설명.

""

5. 비즈니스 검증 및 Q&A 에이전트 (VALIDATION_AGENT 복원)

"QA_AGENT": ""

당신은 특허 전문가이자 삼성카드 내부 보안/기술 검토관입니다.

제안된 당목담글 시스템의 약점을 찾아내고 방어 논리를 구축하는 것이 임무입니다.

[지침]

다음 요소를 포함하여 임원진 대비 예상 질문 3개와 날카로운 모범 답변을 작성하십시오.

1. 프라이버시 방어: "민감한 가족 음성 데이터가 서버에 남는 것 아닌가?" -> On-device 로컬 처리 방식 강조.
2. 차별성 검증: "기존 음성 합성(TTS) 앱이나 녹음기와 무엇이 다른가?" -> 노이즈 보존(Originality)과 릴레이 스티치 엔진의 차별점 강조.
3. 실현 가능성: 긴급 상황 인지(GPS, 스케줄 연동) 로직의 구동 방식 설명.

""

3. utils.py (유틸리티 함수)

```
dmdg > utils.py > save_txt
1  # =====
2  # 유틸리티 함수 설정 (dmdg/utils.py)
3  # =====
4  import os
5
6  def ensure_all_dirs():
7      """모든 데이터 저장 폴더를 dmdg/data 하위에 생성"""
8      base_dir = os.path.join("dmdg", "data")
9      paths = {
10         "base": base_dir,
11         "images": os.path.join(base_dir, "images"),
12         "audio": os.path.join(base_dir, "audio")
13     }
14     for p in paths.values():
15         os.makedirs(p, exist_ok=True)
16     return paths
17
18 def save_txt(path, content):
19     with open(path, "w", encoding="utf-8") as f:
20         f.write(content)
21     print(f"📄 파일 저장 완료: {path}")
```

4. state.py (상태 정의 및 호출 함수)

```
dmdg > state.py > ask_llm
1  # =====
2  # 상태 정의 및 호출 함수 설정 (dmdg/state.py)
3  # =====
4  from typing import TypedDict
5  from .config import groq_client, gemini_client
6
7  class AgentState(TypedDict):
8      topic: str
9      raw_text: str
10     summary: str
11     podcast_script: str
12     audio_path: str
13     image_prompt: str
14     image_path: str
15     ppt_path: str
16     questions: str
```

```

def ask_llm(system_prompt, input_text):
    """페르소나 기반 LLM 호출 엔진 (Gemini -> Groq 백업)"""
    try:
        full_prompt = f"[역할]:\n{system_prompt}\n\n[데이터]:\n{input_text}"
        response = gemini_client.models.generate_content(
            model='gemini-2.0-flash', contents=full_prompt
        )
        return response.text
    except Exception:
        completion = groq_client.chat.completions.create(
            model="llama-3.3-70b-versatile",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": input_text}
            ]
        )
        return completion.choices[0].message.content

```

5. nodes.py (에이전트 노드 정의)

```

dmdg > nodes.py > ...
1  # =====
2  # 에이전트 노드 정의 설정 (dmdg/nodes.py)
3  # =====
4  import os
5  import requests
6  import urllib.parse
7  from gtts import gTTS
8  from pptx import Presentation
9  from pptx.util import Inches, Pt
10 from .state import AgentState, ask_llm
11 from .personas import PERSONA_CONFIG
12 from .utils import ensure_all_dirs, save_txt

```

```

def search_node(state: AgentState):
    print("⌚ [1/8] 검색 중...")
    state['raw_text'] = f'"{state['topic']}" 관련 소방관 목소리 기부 계획안 데이터...'
    return state

```

```

def summary_node(state: AgentState):
    print("⌚ [2/8] 요약 중...")
    dirs = ensure_all_dirs()
    state['summary'] = ask_llm(PERSONA_CONFIG["SUMMARY_AGENT"], state['raw_text'])
    save_txt(os.path.join(dirs['base'], "1_summary.txt"), state['summary'])
    return state

```



```
def podcast_node(state: AgentState):
    print("⌚ [3/8] 대본 생성 중...")
    dirs = ensure_all_dirs()
    state['podcast_script'] = ask_llm(PERSONA_CONFIG["PODCAST_AGENT"], state['summary'])
    save_txt(os.path.join(dirs['base'], "2_podcast.txt"), state['podcast_script'])
    return state
```

```
def audio_node(state: AgentState):
    print("⌚ [4/8] 오디오 합성 중...")
    dirs = ensure_all_dirs()
    path = os.path.join(dirs['audio'], "3_audio.mp3")
    tts = gTTS(text=state['podcast_script'][:1000], lang='ko')
    tts.save(path)
    state['audio_path'] = path
    return state
```

```
def image_node(state: AgentState):
    print("⌚ [5/8] AI 이미지 생성 중...")
    dirs = ensure_all_dirs()
    state['image_prompt'] = ask_llm(PERSONA_CONFIG["IMAGE_AGENT"], state['podcast_script'])
    encoded = urllib.parse.quote(state['image_prompt'])
    url = f"https://image.pollinations.ai/prompt/{encoded}?width=1024&height=768&nologo=true"
    path = os.path.join(dirs['images'], "4_thumb.png")
    res = requests.get(url)
    with open(path, 'wb') as f: f.write(res.content)
    state['image_path'] = path
    return state
```

```
def ppt_node(state: AgentState):
    print("⌚ [6/8] PPT 제작 중...")
    dirs = ensure_all_dirs()
    prs = Presentation()
    ppt_content = ask_llm(PERSONA_CONFIG["PPT_AGENT"], state['summary'])
    slide = prs.slides.add_slide(prs.slide_layouts[6])
    # (여기에 이전의 텍스트 상자 및 이미지 삽입 로직 포함)
    path = os.path.join(dirs['base'], "5_proposal.pptx")
    prs.save(path)
    state['ppt_path'] = path
    return state
```

```
def qa_node(state: AgentState):
    print("⌚ [7/8] Q&A 생성 중...")
    dirs = ensure_all_dirs()
    state['questions'] = ask_llm(PERSONA_CONFIG["QA_AGENT"], state['summary'])
    save_txt(os.path.join(dirs['base'], "6_qa.txt"), state['questions'])
    return state
```

6. main.py (LangGraph 조립 및 실행)

```
dmdg > 🚀 main.py > ...
1  # =====
2  # LangGraph 조립 설정 및 실행 (dmdg/main.py)
3  # =====
4  from langgraph.graph import StateGraph, END
5  from .nodes import *
6  from .state import AgentState
7
```

```
def build_graph():
    workflow = StateGraph(AgentState)

    workflow.add_node("search", search_node)
    workflow.add_node("summary", summary_node)
    workflow.add_node("podcast", podcast_node)
    workflow.add_node("audio", audio_node)
    workflow.add_node("image", image_node)
    workflow.add_node("ppt", ppt_node)
    workflow.add_node("qa", qa_node)

    workflow.set_entry_point("search")
    workflow.add_edge("search", "summary")
    workflow.add_edge("summary", "podcast")
    workflow.add_edge("podcast", "audio")
    workflow.add_edge("audio", "image")
    workflow.add_edge("image", "ppt")
    workflow.add_edge("ppt", "qa")
    workflow.add_edge("qa", END)

    return workflow.compile()
```

```
if __name__ == "__main__":
    print("🚀 분리된 모듈 기반 멀티 에이전트 시스템 가동")
    app = build_graph()
    app.invoke({"topic": "소방관 아빠의 목소리 기부 플랫폼"})
```

이렇게 사용

1. 모든 파일을 dmdg라는 하나의 폴더 안에 넣는다.
2. 터미널에서 dmdg 폴더가 있는 위치로 이동.
3. python -m dmdg.main 명령어로 실행.

이 구조는 **실무 에이전트 개발의 표준**과 같다. 이제 기능 추가는 nodes.py에서, 말투 수정은 personas.py에서만 하면 되므로 작업 속도가 훨씬 빨라질 것

```
(pylocal) PS C:\Users\user\Downloads\dangmok\pylocal\dmdg> python -m dmdg.main
🚀 분리된 모듈 기반 멀티 에이전트 시스템 가동
⌚ [1/8] 검색 중...
⌚ [2/8] 요약 중...
📄 파일 저장 완료: dmdg\data\1_summary.txt
⌚ [3/8] 대본 생성 중...
📄 파일 저장 완료: dmdg\data\2_podcast.txt
⌚ [4/8] 오디오 합성 중...
⌚ [5/8] AI 이미지 생성 중...
⌚ [6/8] PPT 제작 중...
⌚ [7/8] Q&A 생성 중...
📄 파일 저장 완료: dmdg\data\6_qa.txt
(pylocal) PS C:\Users\user\Downloads\dangmok\pylocal\dmdg> []
```

```
▼ dmdg\data
  ▼ audio
    🔊 3_audio.mp3
  ▼ images
    🖼️ 4_thumb.png
    ≡ 1_summary.txt
    ≡ 2_podcast.txt
    ≡ 5_proposal.pptx
    ≡ 6_qa.txt
```